

Universidad Central del Ecuador

Arquitectura de Software

Título: Abstract Factory

Curso: SIR – S7- 001

Integrantes: Diego Lema
Jefferson Pilataxi
Juan Tulcanaza





Contents

Manual de Implementación y portabilidad	3
Introducción	3
Requisitos del sistema.....	3
Software necesario	3
Requisitos mínimos sugeridos.....	3
Tecnologías utilizadas.....	3
Arquitectura del sistema (contenedores).....	4
Diagrama de arquitectura (Insertar imagen)	4
<i>Descripción del flujo:</i>	4
Justificación de las bases de datos.....	4
PostgreSQL (Relacional).....	4
<i>Tablas principales:</i>	5
MongoDB (Documental).....	5
<i>Colección principal:</i>	5
Patrones de diseño implementados.....	5
Abstract Factory	5
DAO (Data Access Object)	5
DTO (Data Transfer Object).....	6
Configuración del entorno (.env)	6
Creación del archivo	6
Contenido esperado del .env	6
Instalación y despliegue (portabilidad a otra PC)	7
Repositorios publicados en Docker Hub:.....	7
PASOS DE EJECUCIÓN (en otro computador)	7
ACCESOS DEL SISTEMA.....	8
DETENER EL SISTEMA	8
Guía de uso del sistema (para evaluación).....	8
Registro e inicio de sesión	8
CRUD de tareas	8
Endpoints del backend (Swagger)	8



Autenticación	8
Tareas	9
Reporte combinado.....	9
Evidencia del reporte combinado	9
Explicación técnica	9
Ejemplo de salida.....	9
Backups de bases de datos.....	9
Generación de dump de PostgreSQL.....	9
Generación de dump de MongoDB.....	10
Restauración de backups.....	10
Restaurar PostgreSQL	10
Restaurar MongoDB	10
Solución de problemas comunes	10
Puertos ocupados	10
Contenedores no levantan	10
Rebuild completo	10
Entregables incluidos y excluidos	10
Conclusión	11
Anexos	12
Diagramas del sistema.....	12
Diagrama de arquitectura en capas	12
Diagrama de componentes	13
Diagrama de despliegue (Docker).....	14
Diagrama de flujo del proceso de login.....	15
Diagrama de flujo del proceso de gestión de tareas	17
Función de la Aplicación	18

Manual de Implementación y portabilidad

INTRODUCCIÓN

Este documento describe la implementación, despliegue y portabilidad del Sistema de Gestión de Tareas, construido bajo una arquitectura distribuida con contenedores Docker, dos orígenes de datos (PostgreSQL y MongoDB), y patrones de diseño Abstract Factory, DAO y DTO.

El sistema permite registro e inicio de sesión con correo institucional @uce.edu.ec, CRUD de tareas y generación de un reporte combinado entre ambas bases.

Requisitos del sistema

SOFTWARE NECESARIO

- Docker Desktop (Windows / Linux / Mac)
- Docker Compose (incluido en Docker Desktop)
- Navegador web moderno (Chrome / Edge / Firefox)

REQUISITOS MÍNIMOS SUGERIDOS

- RAM: 4 GB mínimo (8 GB recomendado)
- Procesador: 2 núcleos o más
- Espacio libre: 1 GB

Tecnologías utilizadas

Componente	Tecnología
Backend	FastAPI (Python)
Frontend	HTML/CSS/JS + Nginx
Base de datos 1	PostgreSQL
Base de datos 2	MongoDB
Contenerización	Docker + Docker Compose
Autenticación	JWT

Patrón	Abstract Factory
Acceso a datos	DAO
Transferencia / validación	DTO (Pydantic)

Arquitectura del sistema (contenedores)

El sistema se despliega mediante **Docker Compose** y se divide en 4 contenedores:

Contenedor	Puerto	Función
frontend	8080	Interfaz web
backend	8000	API REST + Swagger
postgres	5432	Datos relacionales (usuarios, tareas)
mongo	27017	Datos NoSQL (logs, auditoría)

DIAGRAMA DE ARQUITECTURA (INSERTAR IMAGEN)

Incluir una imagen en el PDF desde:
</diagrams/architecture.png>

Descripción del flujo:

1. El usuario interactúa con el Frontend (Nginx).
2. Frontend consume endpoints del Backend (FastAPI).
3. Backend guarda usuarios y tareas en PostgreSQL.
4. Backend registra logs de acciones en MongoDB.
5. Backend genera un reporte combinado consultando ambas bases.

Justificación de las bases de datos

POSTGRESQL (RELACIONAL)

Se utiliza PostgreSQL porque:

- Maneja datos estructurados con relaciones (usuarios → tareas)
- Asegura integridad y consistencia mediante transacciones (ACID)
- Es ideal para operaciones CRUD críticas

Tablas principales:

- users (email, password_hash, created_at)
- tasks (title, description, status, owner_id)

MONGODB (DOCUMENTAL)

Se utiliza MongoDB porque:

- Permite almacenar historial y auditoría en forma de documentos flexibles
- Crece fácilmente conforme se generan logs
- Facilita consultas de “última acción” por tarea

Colección principal:

- task_logs (task_id, owner_id, action, detail, timestamp)

Patrones de diseño implementados

ABSTRACT FACTORY

Se usa para instanciar objetos DAO de acuerdo a cada origen de datos:

- PostgresFactory: crea UserDAO y TaskDAO
- MongoFactory: crea LogDAO

Ventajas:

- Reduce acoplamiento entre servicios y bases de datos
- Permite escalabilidad (nuevos orígenes sin reescribir lógica)

DAO (DATA ACCESS OBJECT)

Encapsula el acceso a datos:

- PostgresUserDAO → operaciones de usuarios en PostgreSQL
- PostgresTaskDAO → operaciones de tareas en PostgreSQL
- MongoLogDAO → registro de logs en MongoDB

DTO (DATA TRANSFER OBJECT)

Se usa para validar datos de entrada/salida mediante Pydantic:

- RegisterDTO, LoginDTO
- TaskCreateDTO, TaskUpdateDTO
- ReportDTO

Incluye validación de:

- correo institucional @uce.edu.ec
- contraseña mínima
- longitud de título y descripción
- estados permitidos: PENDING, IN_PROGRESS, DONE

Configuración del entorno (.env)

Crear el archivo .env desde .env.example:

CREACIÓN DEL ARCHIVO

En la raíz del proyecto:

```
cp .env.example .env
```

CONTENIDO ESPERADO DEL .ENV

```
POSTGRES_USER=appuser
```

```
POSTGRES_PASSWORD=apppass
```

```
POSTGRES_DB=taskdb
```

```
POSTGRES_HOST=postgres
```

```
POSTGRES_PORT=5432
```

```
MONGO_INITDB_ROOT_USERNAME=root
```

```
MONGO_INITDB_ROOT_PASSWORD=example
```

```
MONGO_HOST=mongo
```

```
MONGO_PORT=27017
```

```
MONGO_DB=tasklogs
```

```
JWT_SECRET=super_secret_key
```

```
JWT_EXPIRES_MIN=60
```

```
CORS_ORIGINS=http://localhost:8080
```

Instalación y despliegue (portabilidad a otra PC)

Para mejorar la portabilidad del sistema, las imágenes del backend y frontend fueron publicadas en Docker Hub, permitiendo ejecutar el sistema en cualquier computador con Docker instalado, sin necesidad de construir localmente (--build). El despliegue se realiza descargando las imágenes y levantando los contenedores con Docker Compose.

REPOSITORIOS PUBLICADOS EN DOCKER HUB:

- juantulcanaza/taskmanager-backend:latest
- juantulcanaza/taskmanager-frontend:latest

PASOS DE EJECUCIÓN (EN OTRO COMPUTADOR)

1. Instalar **Docker Desktop** (incluye Docker Compose).
2. Copiar o descomprimir el proyecto en el computador destino.
3. Abrir una terminal en la carpeta raíz del proyecto (donde se encuentra docker-compose.yml).
4. Crear el archivo .env a partir del ejemplo:

Windows (PowerShell):

```
copy .env.example .env
```

Linux/Mac:

```
cp .env.example .env
```

5. Descargar las imágenes desde Docker Hub:

```
docker compose pull
```

6. Levantar el sistema:

```
docker compose up
```

7. Verificar que existan 4 contenedores activos:

```
docker compose ps
```


ACCESOS DEL SISTEMA

- Frontend: <http://localhost:8080>
- Swagger (Backend): <http://localhost:8000/docs>

DETENER EL SISTEMA

docker compose down

Guía de uso del sistema (para evaluación)

REGISTRO E INICIO DE SESIÓN

En el frontend (puerto 8080), ingresar:

- Email: `usuario@uce.edu.ec`
- Password: `Ejemplo123!`

El sistema valida el dominio `@uce.edu.ec`.
Al iniciar sesión, se genera un token JWT.

CRUD DE TAREAS

Una tarea tiene los campos:

- title: título corto
- description: detalle opcional
- status: PENDING / IN_PROGRESS / DONE

Acciones:

- Crear tarea
- Listar tareas
- Cambiar estado
- Eliminar tarea

Endpoints del backend (Swagger)

AUTENTICACIÓN

- POST `/auth/register`
- POST `/auth/login`

TAREAS

- POST /tasks
- GET /tasks
- PUT /tasks/{id}
- DELETE /tasks/{id}

REPORTE COMBINADO

- GET /reports/tasks-with-last-change

Evidencia del reporte combinado

EXPLICACIÓN TÉCNICA

El reporte combina:

- **PostgreSQL:** lista de tareas del usuario
- **MongoDB:** última acción registrada por tarea (CREATE/UPDATE/DELETE)

EJEMPLO DE SALIDA

```
[  
  {  
    "task_id": "a1c27325-8bd4-41d9-81b8-54f1f79a50f7",  
    "title": "Hola",  
    "status": "PENDING",  
    "last_action": "CREATE",  
    "last_action_at": "2026-01-15T18:49:44.000Z"  
  }  
]
```

Backups de bases de datos

GENERACIÓN DE DUMP DE POSTGRESQL

```
docker compose exec -T postgres pg_dump -U appuser taskdb >  
backups/postgres_dump.sql
```

GENERACIÓN DE DUMP DE MONGODB

```
docker compose exec -T mongo mongodump --archive > backups/mongo_dump.archive
```

Restauración de backups

RESTAURAR POSTGRESQL

```
docker compose exec -T postgres psql -U appuser -d taskdb <
backups/postgres_dump.sql
```

RESTAURAR MONGODB

```
docker compose exec -T mongo mongorestore --archive <
backups/mongo_dump.archive
```

Solución de problemas comunes

PUERTOS OCUPADOS

Si 8080 o 8000 están en uso:

- Cambiar en docker-compose.yml o cerrar el proceso que lo usa.

CONTENEDORES NO LEVANTAN

Ejecutar:

```
docker compose logs -f backend
```

```
docker compose logs -f postgres
```

```
docker compose logs -f mongo
```

REBUILD COMPLETO

```
docker compose down
```

```
docker compose pull
```

```
docker compose up
```

Entregables incluidos y excluidos

Incluye

- Código fuente
- Dockerfiles y docker-compose.yml



- .env.example
- Diagramas
- Backups

No incluye

- .env real con credenciales personales
- node_modules
- .venv
- ejecutables/ensamblados
- dependencias descargadas

Conclusión

En conclusión, el prototipo del Sistema de Gestión de Tareas cumple con los objetivos propuestos, demostrando el uso de múltiples orígenes de datos, una arquitectura en capas bien definida, la aplicación del patrón Abstract Factory y un despliegue basado en contenedores.

Se concluye que el presente sistema tiene, dos orígenes de datos desplegados en contenedores, backend web que implementa Abstract Factory + DAO + DTO, operaciones CRUD completas, reporte combinado de datos de ambas bases.

Anexos

Diagramas del sistema

DIAGRAMA DE ARQUITECTURA EN CAPAS

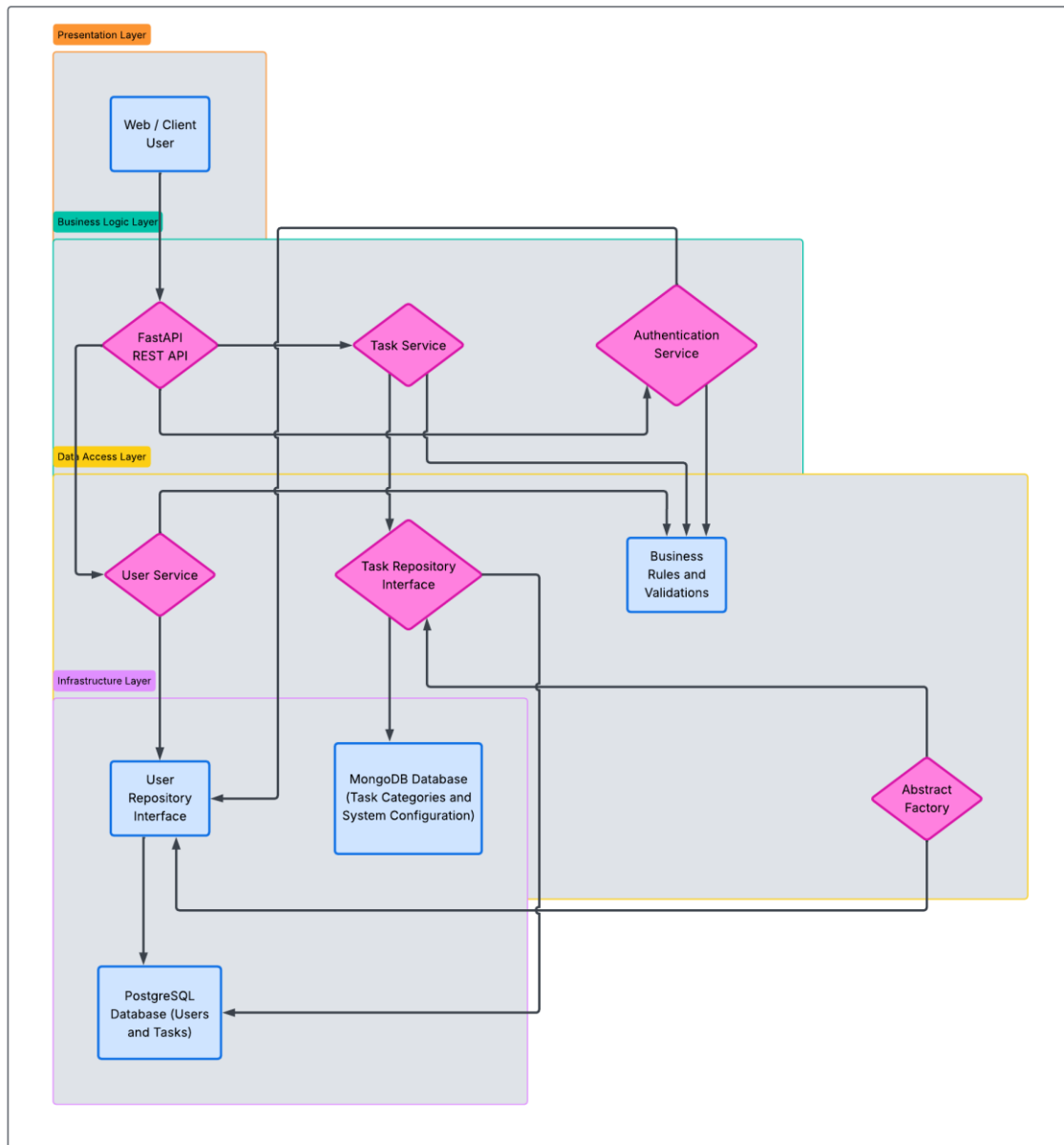


figura 1. Diagrama de arquitectura de capas

El diagrama de arquitectura en capas representa la organización interna del sistema desde un enfoque estructural. En este diagrama se puede observar cómo el sistema está dividido en distintas capas, cada una con responsabilidades bien definidas.

En la parte superior se encuentra la capa de presentación, representada por la API desarrollada con FastAPI, que es el punto de entrada para todas las solicitudes realizadas por los usuarios. Esta capa se encarga de recibir las peticiones, validar datos básicos y delegar la lógica al siguiente nivel.

La capa de negocio se encarga de procesar las reglas del sistema, como la validación de usuarios, la gestión de tareas y el control de acceso. Esta separación permite que la lógica principal del sistema no dependa directamente de la forma en la que se almacenan los datos.

La capa de acceso a datos actúa como intermediaria entre la lógica de negocio y los orígenes de datos. Gracias a esta capa, el sistema puede interactuar tanto con PostgreSQL como con MongoDB de forma transparente.

Finalmente, la capa de infraestructura agrupa los elementos técnicos necesarios para la ejecución del sistema, como las bases de datos y los contenedores Docker. Este enfoque facilita el mantenimiento y la escalabilidad del sistema.

DIAGRAMA DE COMPONENTES

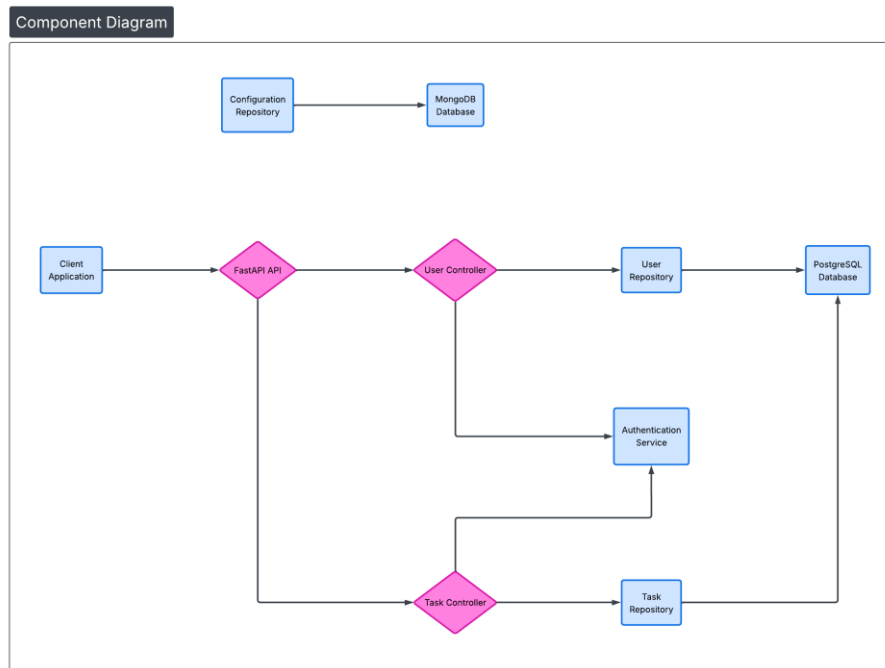


figura 2. Diagrama de componentes

El diagrama de componentes muestra cómo se divide el sistema internamente en módulos funcionales y cómo estos interactúan entre sí. A diferencia del diagrama en capas, este se enfoca más en los componentes específicos que conforman la aplicación.

En este diagrama se observa que el usuario interactúa con la API FastAPI, la cual dirige las solicitudes hacia los controladores correspondientes, como el controlador de usuarios o el controlador de tareas. Cada controlador se encarga de manejar una funcionalidad específica del sistema.

Los controladores no acceden directamente a las bases de datos, sino que utilizan repositorios definidos a través del patrón Abstract Factory. Esto permite que el sistema pueda cambiar o extender sus orígenes de datos sin modificar la lógica principal.

Además, el diagrama evidencia la separación entre los repositorios que acceden a PostgreSQL y aquellos que acceden a MongoDB, reforzando el desacoplamiento del sistema y mejorando su mantenibilidad.

DIAGRAMA DE DESPLIEGUE (DOCKER)

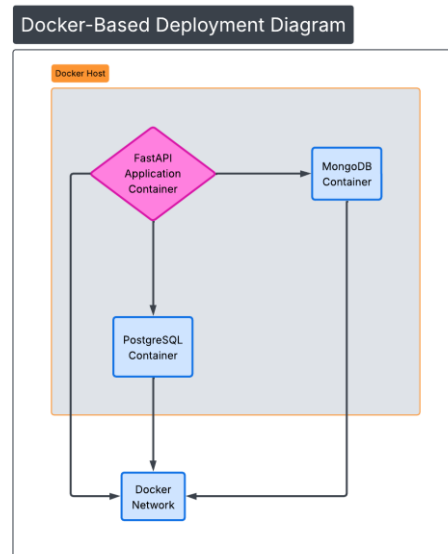


figura 3. Diagrama de despliegue (Docker)

El diagrama de despliegue ilustra cómo el sistema se ejecuta en un entorno real utilizando contenedores Docker. Este diagrama permite entender de qué manera los distintos componentes del sistema se distribuyen en contenedores independientes.

En el diagrama se observa un contenedor que ejecuta la aplicación FastAPI, el cual se comunica con los contenedores de PostgreSQL y MongoDB a través de una red interna. Esta separación asegura que cada componente tenga su propio entorno de ejecución y pueda ser administrado de forma independiente.

Docker Compose se encarga de la orquestación de los contenedores, permitiendo iniciar, detener y configurar todo el sistema de manera sencilla. Este enfoque facilita tanto el

desarrollo como las pruebas del sistema, ya que el entorno se mantiene consistente en diferentes máquinas.

DIAGRAMA DE FLUJO DEL PROCESO DE LOGIN

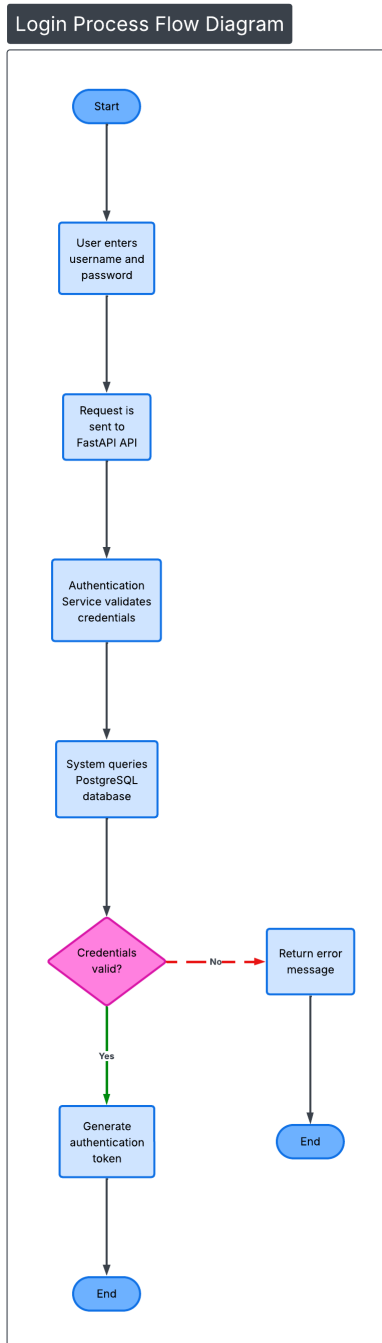


figura 4. Diagrama de flujo de proceso login

El diagrama de flujo del proceso de login describe paso a paso cómo se lleva a cabo la autenticación de un usuario dentro del sistema. El flujo inicia cuando el usuario ingresa sus credenciales en la interfaz del sistema.

Posteriormente, la API recibe los datos y los envía a la capa de negocio para su validación. En este punto, el sistema verifica si el usuario existe y si las credenciales ingresadas son correctas.

En caso de que la autenticación falle, el sistema devuelve un mensaje de error indicando que las credenciales no son válidas. Si la autenticación es exitosa, el sistema permite el acceso del usuario a las funcionalidades disponibles.

Este diagrama es importante porque permite entender claramente el control de acceso al sistema y cómo se gestionan las validaciones de seguridad de forma ordenada.

DIAGRAMA DE FLUJO DEL PROCESO DE GESTIÓN DE TAREAS

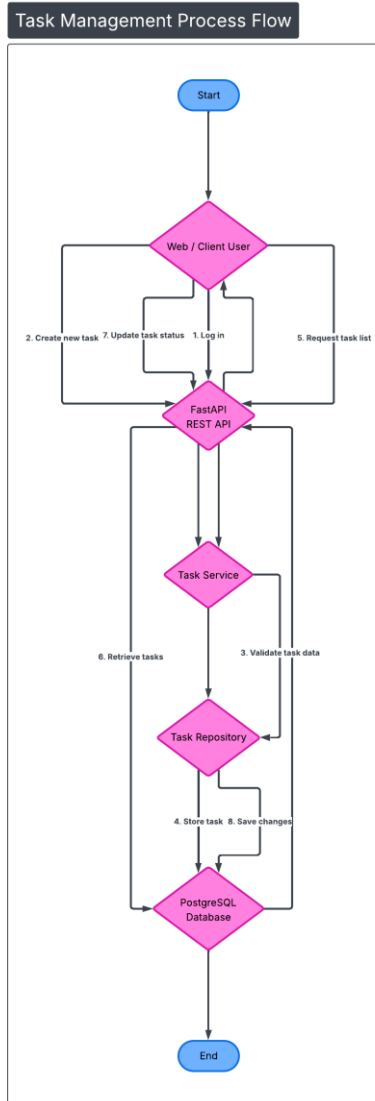


figura 5. Diagrama de flujos de proceso gestión de tareas

El diagrama de flujo del proceso de gestión de tareas representa el ciclo principal de uso del sistema por parte del usuario una vez que ha iniciado sesión. Este diagrama permite visualizar la interacción entre el usuario, la API, la capa de servicios y la base de datos PostgreSQL.

El flujo inicia cuando el usuario accede al sistema y se autentica correctamente. Una vez validado, el usuario puede crear una nueva tarea ingresando la información correspondiente, como el título, la descripción y el estado inicial.

La solicitud es enviada a la API, la cual redirige la información al servicio de tareas. Este servicio se encarga de validar los datos ingresados, asegurando que la información cumpla con las reglas del sistema antes de ser almacenada.

Posteriormente, el repositorio de tareas guarda la información en la base de datos PostgreSQL, donde se mantiene la persistencia de las tareas asociadas al usuario.

En una segunda etapa del flujo, el usuario puede solicitar la lista de tareas registradas. El sistema consulta la base de datos y retorna la información correspondiente para su visualización.

Finalmente, cuando el usuario decide actualizar el estado de una tarea (por ejemplo, marcarla como completada), el sistema procesa la solicitud y guarda los cambios nuevamente en PostgreSQL, asegurando que la información se mantenga actualizada.

Este diagrama es fundamental porque muestra de forma clara cómo interactúan los distintos componentes del sistema durante el uso normal de la aplicación y permite entender el flujo completo desde la creación hasta la actualización de una tarea.

Función de la Aplicación

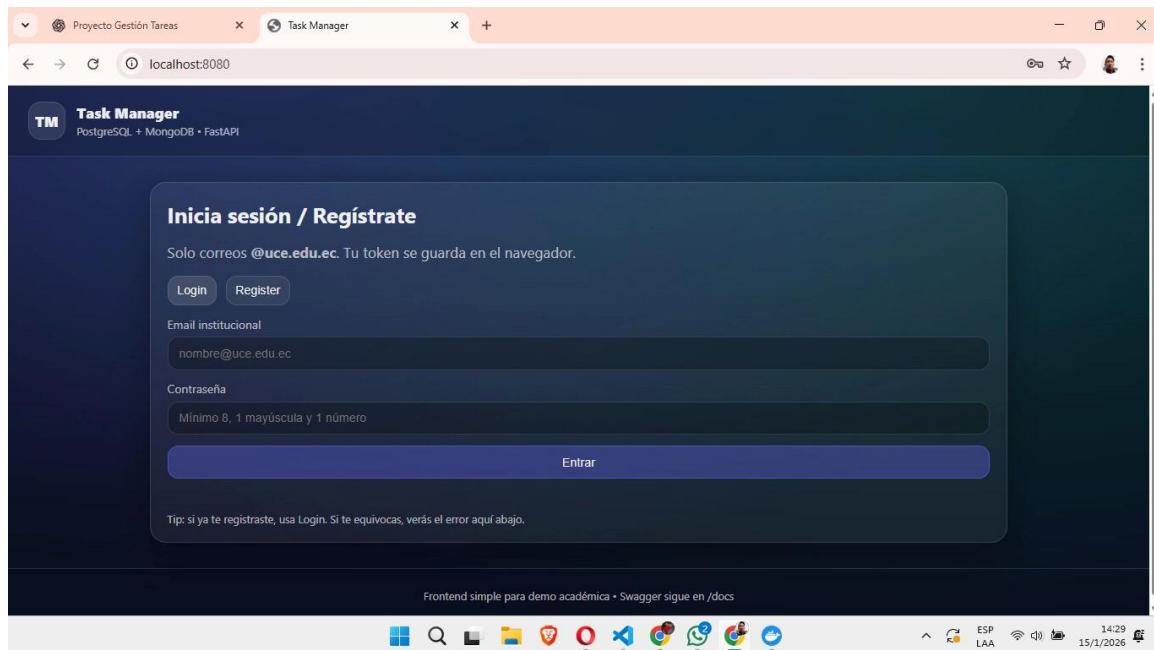


figura 6. Pantalla de Registrate.

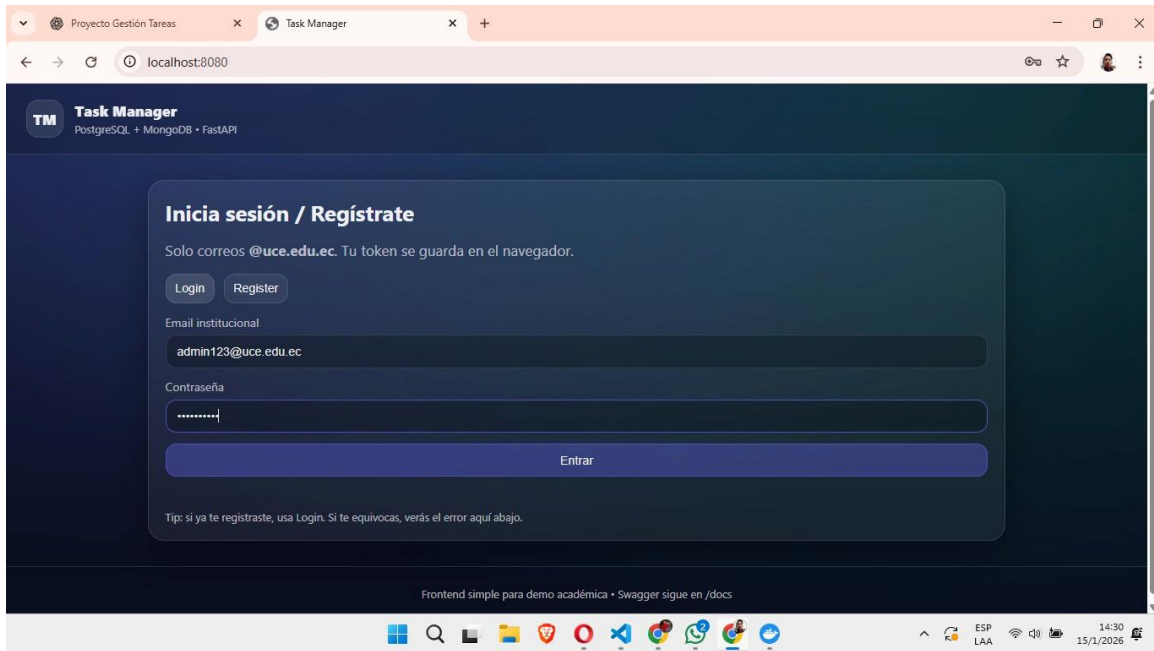


figura 7. Pantalla de Registrate, con datos.

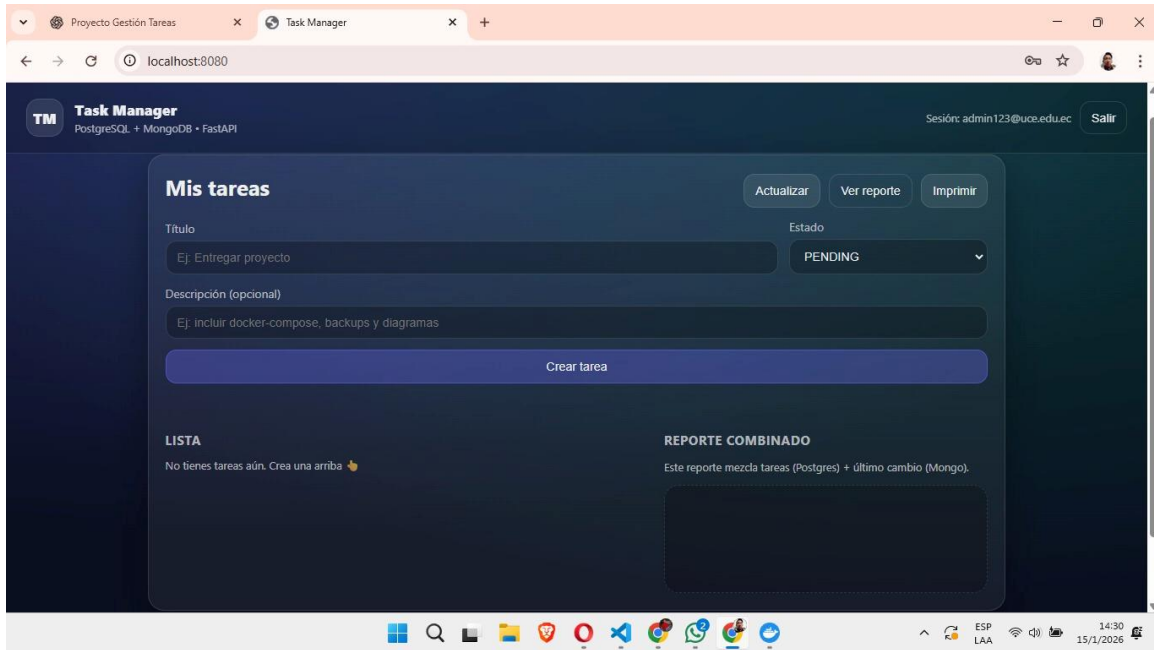


figura 8. Pantalla con usuario logueado.

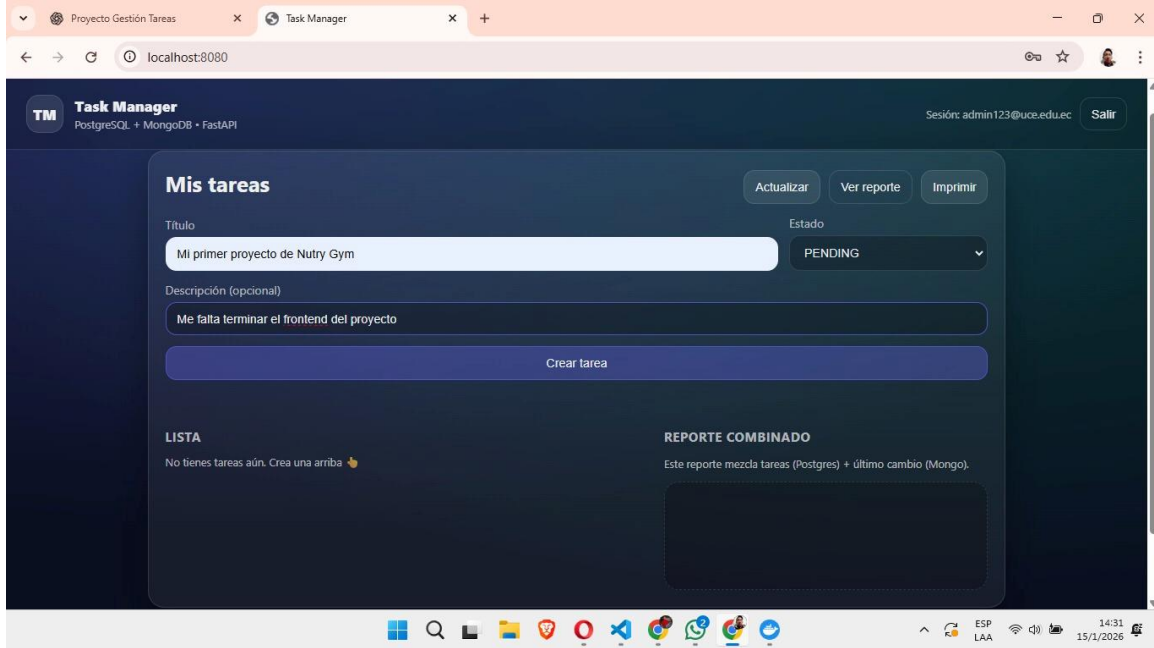


figura 8. Pantalla de inicio con datos llenados.

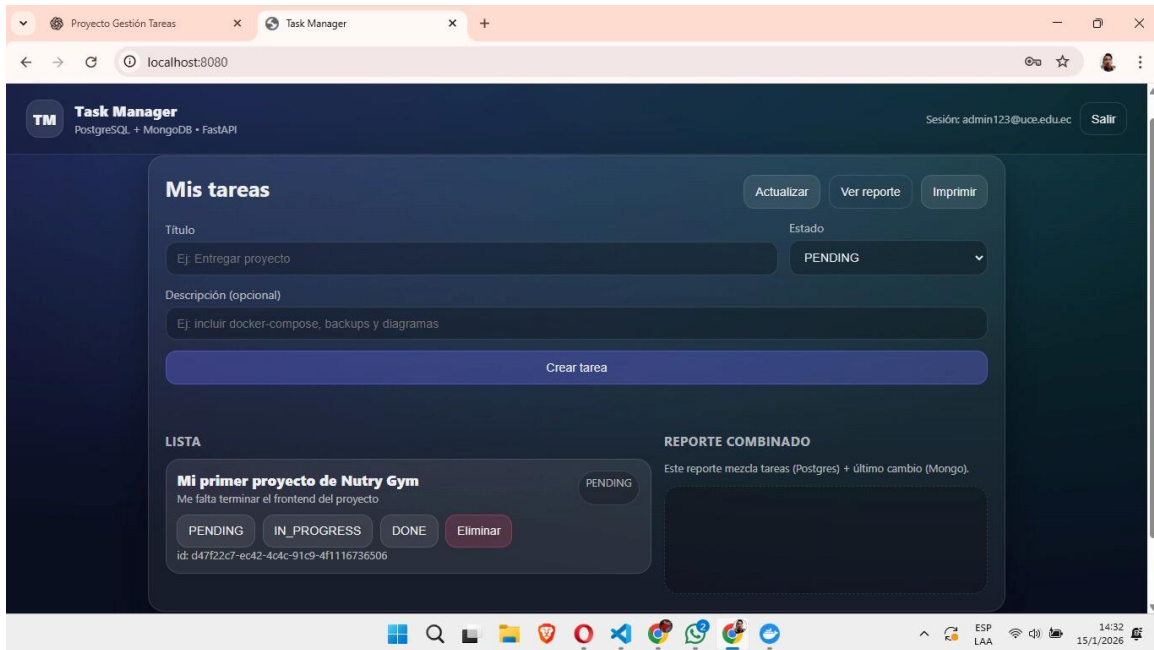


figura 9. Pantalla de inicio, creando tarea.

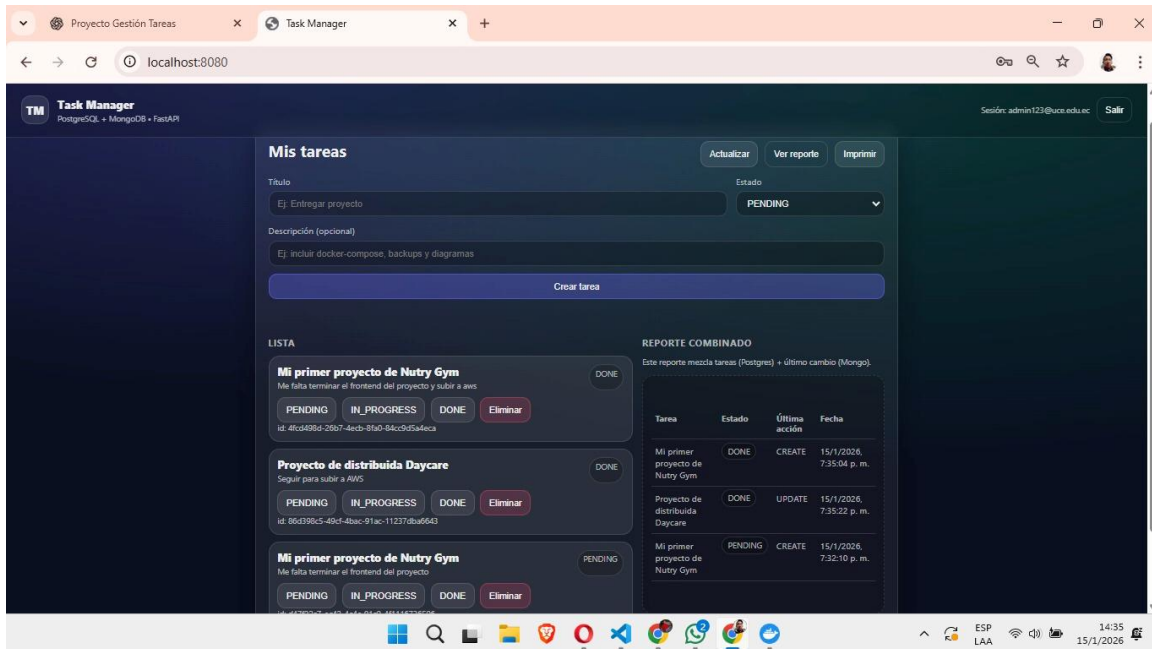


figura 10. Pantalla de inicio, dando click en ver reporte.

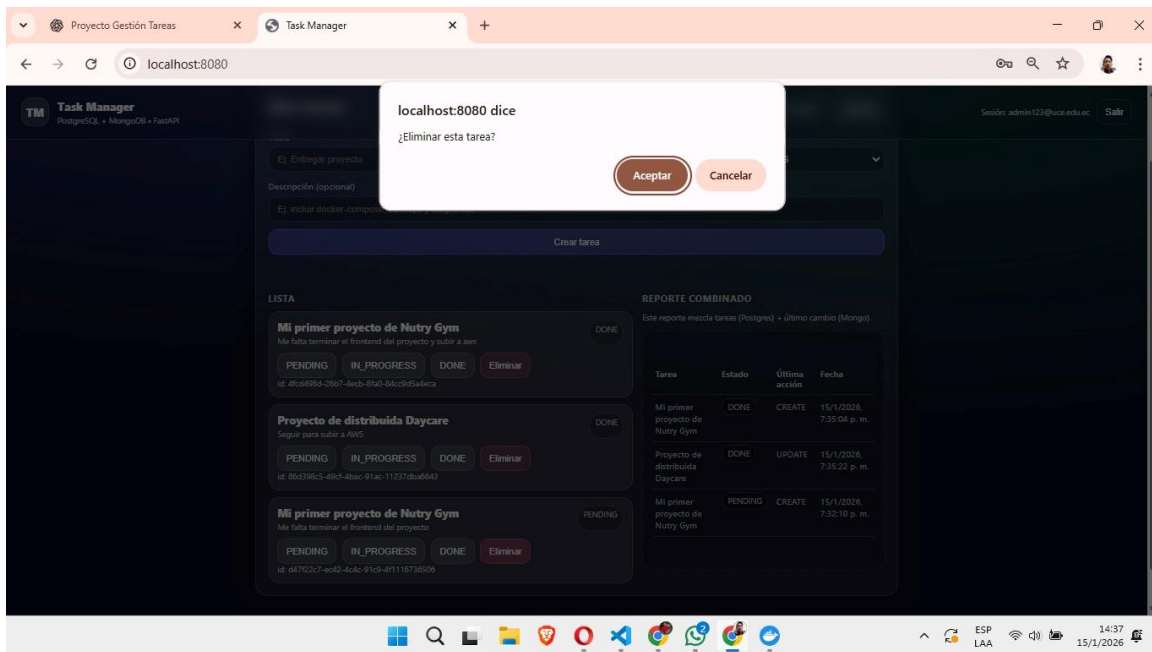


figura 11. Pantalla de inicio eliminando usuario.

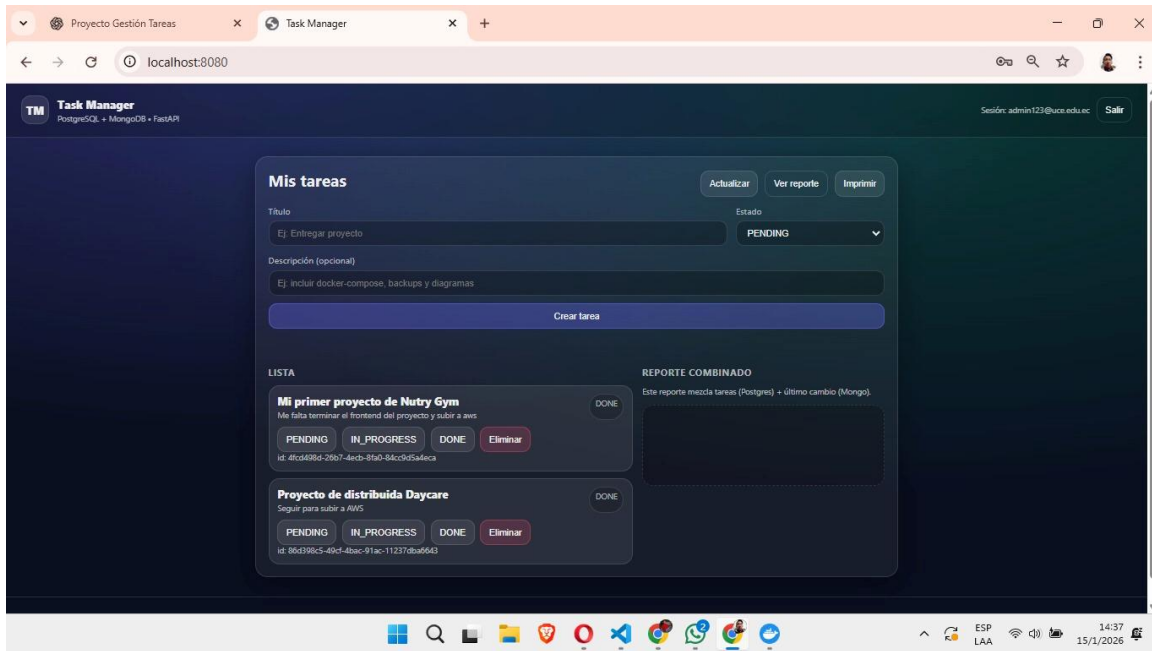


figura 12. Pantalla de inicio, usuario eliminado.

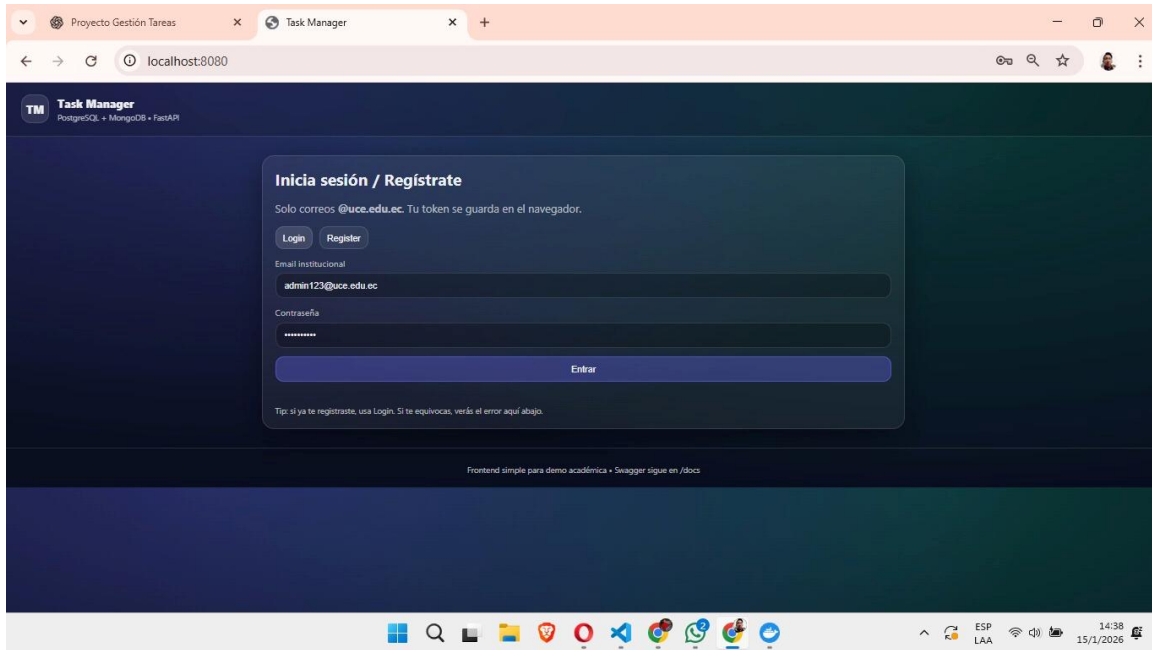


figura 13. Pantalla login.

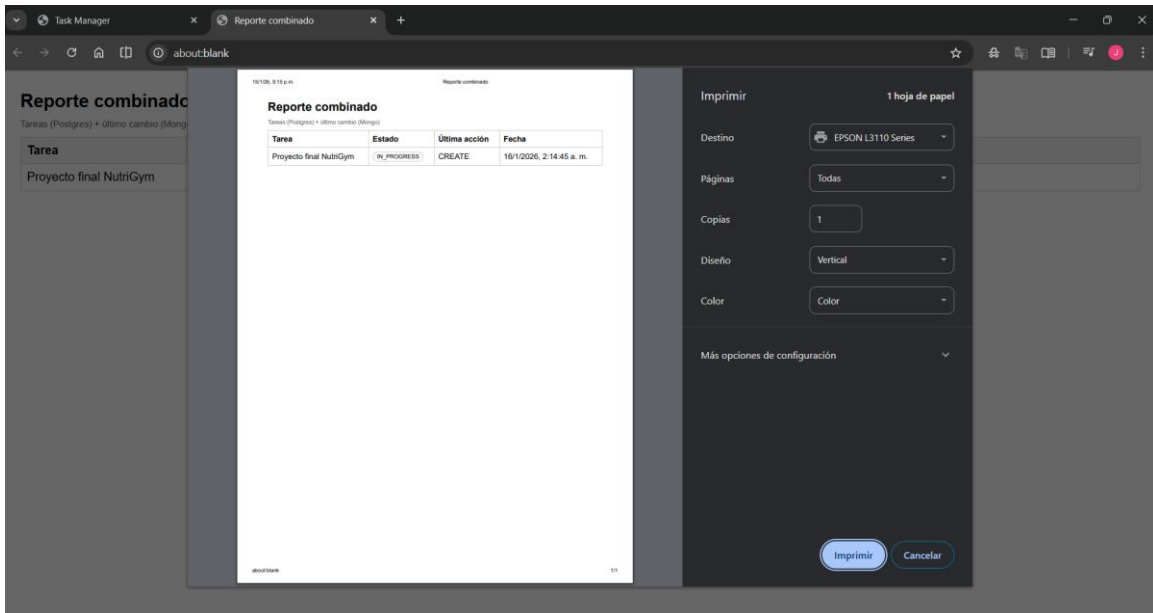


figura 14. Pantalla de Descarga del reporte combinado.

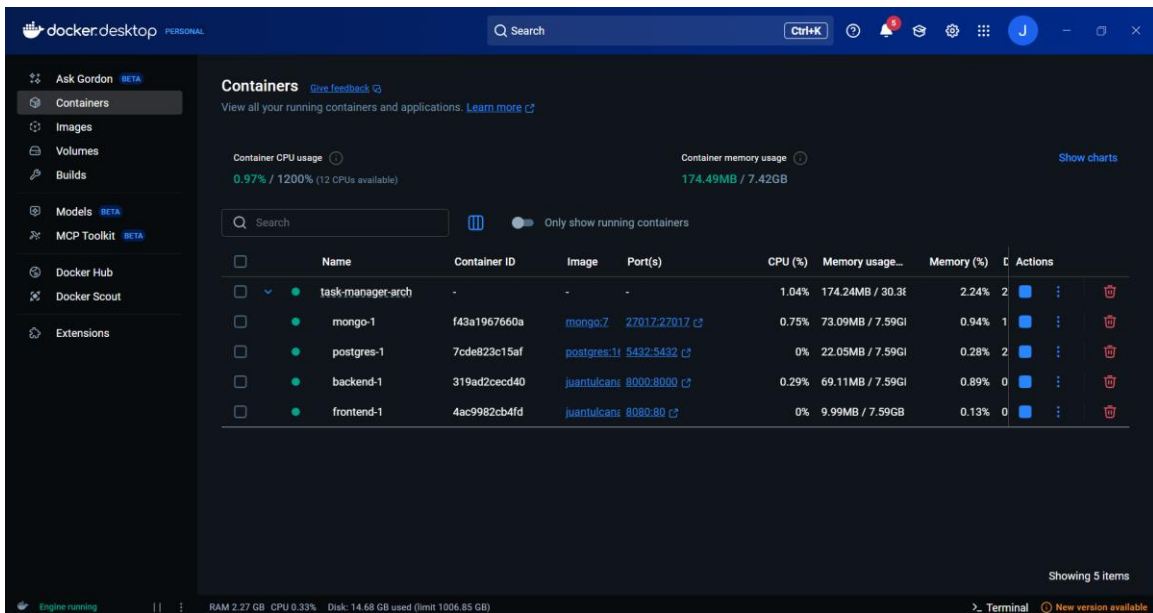


figura 15. Pantalla de Docker Desktop con los contenedores corriendo.

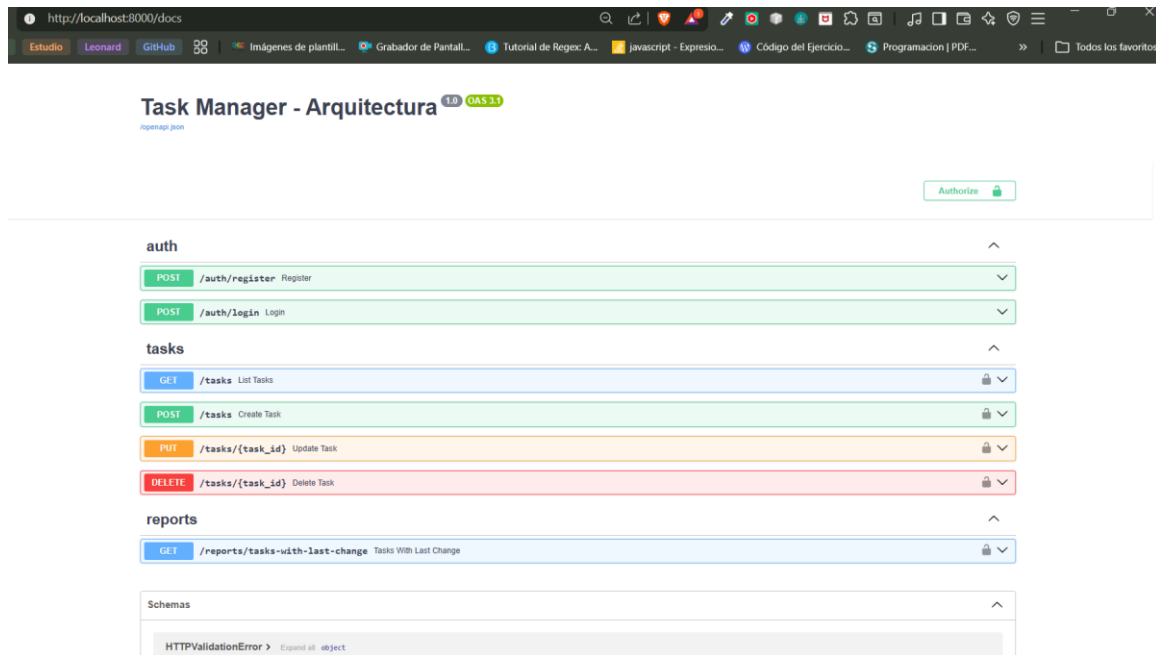


figura 16. Pantalla de del microservicio corriendo.